

Résumé

Le projet *Data Breach & Threat Intelligence Monitoring Platform* propose une plateforme de veille Data Breach & Threat Intelligence Monitoring Platform dédiée à la détection des expositions publiques de données sensibles. La plateforme agrège et corrèle plusieurs sources ouvertes : **GitHub** pour les fuites techniques de secrets, **Telegram** pour la veille communautaire et CVE, **Google Alerts** pour la dimension OSINT et médiatique, ainsi qu'un module complémentaire de *Threat Intelligence* consacré aux flux CVE et à la production de tableaux de bord BI.

L'approche retenue est méthodologique : chaque source est configurée à partir de fichiers de référence, les données collectées sont normalisées, analysées, puis filtrées par une *detection policy* avant d'être classifiées par catégorie de risque et par niveau de criticité. Les signaux qualifiés sont ensuite stockés, visualisés dans des tableaux de bord interactifs et exploités par l'analyste pour la prise de décision.

Table des matières

Résumé	1
1 Contexte, objectifs et périmètre du projet	4
1.1 Contexte général	4
1.2 Objectif global	4
1.3 Périmètre fonctionnel	5
1.4 Public cible	5
1.5 Vision méthodologique	5
2 Architecture générale et méthodologie de veille	6
2.1 Vue d'ensemble de l'architecture	6
2.2 Plateforme principale de monitoring	7
2.3 Module BI Threat Intelligence	7
2.4 Configuration des sources	8
2.5 Contrôle des sources	8
3 Sources de données et stratégie de collecte	10
3.1 Vue d'ensemble des sources	10
3.2 GitHub Monitoring	11
3.2.1 Rôle de la source	11
3.2.2 Mécanisme d'accès	11
3.2.3 Organisation des requêtes	11
3.2.4 Exemple de résultat GitHub	12
3.3 Telegram Monitoring	14
3.3.1 Rôle de la source	14
3.3.2 Canaux suivis	14
3.3.3 Logique de traitement	14
3.3.4 Exemple de signal Telegram	15
3.4 Google Alerts Monitoring	17
3.4.1 Rôle de la source	17
3.4.2 Configuration des alertes	17
3.4.3 Logique d'exploitation	18
3.5 CVE Feed et BI Threat Intelligence	19

3.5.1	Positionnement du module	19
3.5.2	Architecture du pipeline CVE	19
3.6	Rôle des tableaux de bord	20
3.7	Tableaux de bord Power BI	20
3.7.1	Executive Overview	21
3.7.2	Time and Age Analysis	21
3.7.3	Severity and Impact Analysis	22
3.8	Apport analytique du module BI	23
3.9	Synthèse des résultats obtenus	23
4	Traitement, détection et classification des risques	25
4.1	Méthodologie de collecte	25
4.2	Limites de volumétrie	25
4.3	Déduplication	25
4.4	Vue d'ensemble des passages de collecte	26
4.5	Normalisation des données	26
4.6	Analyse du contenu	26
4.7	Filtrage	27
4.8	Detection policy	27
4.9	Classification des risques	27
4.10	Gestion des faux positifs	28
	Conclusion générale	29

Contexte, objectifs et périmètre du projet

Contexte général

Le projet *Data Breach & Threat Intelligence Monitoring Platform* a été conçu pour répondre à un constat simple : une grande quantité d'informations sensibles apparaît quotidiennement sur des espaces publics. Il peut s'agir de secrets techniques, d'identifiants, de mentions de fuites de données, de vulnérabilités publiées ou encore de signaux communautaires liés à des incidents cyber.

Ces expositions sont rarement intentionnelles, mais elles représentent un risque de sécurité important. Un secret oublié dans un dépôt public, une annonce de fuite sur un canal OSINT ou une vulnérabilité critique récemment publiée peuvent constituer des éléments exploitables par un attaquant.

Dans ce contexte, le projet vise à fournir une plateforme de veille défensive capable de centraliser, qualifier et restituer ces signaux à un analyste.

Objectif global

L'objectif principal de la plateforme est de collecter, normaliser, filtrer et présenter des signaux cyber issus de sources ouvertes et autorisées. La solution permet de gagner du temps sur la collecte manuelle, de réduire la dispersion de l'information et de fournir un cadre cohérent pour prioriser les détections.

Le projet ne vise pas à mener des actions offensives. Il s'inscrit dans une logique de **veille défensive**, d'**OSINT** et de **Threat Intelligence**. Son rôle est d'aider l'analyste à identifier rapidement les signaux pertinents, à comprendre leur criticité et à suivre leur évolution.

Périmètre fonctionnel

Du point de vue fonctionnel, la plateforme assure trois grands rôles.

- **Collecte multi-sources** : interrogation de GitHub pour les fuites techniques, Telegram pour les signaux communautaires, Google Alerts pour la veille médiatique et les flux CVE pour l'analyse des vulnérabilités.
- **Traitement et qualification** : analyse de contenu, application d'une *detection policy*, déduplication, redaction et classification selon le niveau de risque.
- **Restitution analytique** : visualisation des signaux dans des tableaux de bord permettant à l'analyste de suivre l'état des sources, les résultats collectés et les tendances observées.

Public cible

Ce périmètre s'adresse principalement à :

- un **analyste SOC** chargé de la veille quotidienne ;
- un **responsable sécurité** souhaitant suivre des indicateurs de risque ;
- un **enseignant ou chercheur** voulant illustrer une chaîne de veille OSINT complète ;
- une équipe projet souhaitant expérimenter une architecture de collecte, de traitement et de visualisation cyber.

Vision méthodologique

Ce projet traite la veille cyber comme un processus méthodologique et non comme une simple accumulation de résultats. La plateforme repose sur quatre principes directeurs :

- **séparation des responsabilités** : chaque source dispose de son propre collecteur ;
- **configuration externalisée** : les requêtes, canaux et alertes sont décrits dans des fichiers dédiés ;
- **filtrage explicite** : les règles de validation, de scoring et de redaction sont centralisées ;
- **approche défensive** : les données sensibles ne sont pas conservées en clair.

Cette vision permet de construire une plateforme plus lisible, plus maintenable et plus facilement auditable.

Architecture générale et méthodologie de veille

Vue d'ensemble de l'architecture

La plateforme repose sur une architecture modulaire organisée autour d'un pipeline de veille. Ce pipeline couvre les étapes principales suivantes : configuration des sources, collecte, normalisation, analyse, filtrage, rédaction, stockage et restitution à l'analyste.

La figure 2.1 synthétise l'écosystème complet du projet en distinguant deux sous-systèmes complémentaires. Le premier correspond à la plateforme principale de monitoring, orientée vers la détection d'expositions publiques. Le second correspond au module *BI Threat Intelligence*, orienté vers l'exploitation analytique des flux CVE.

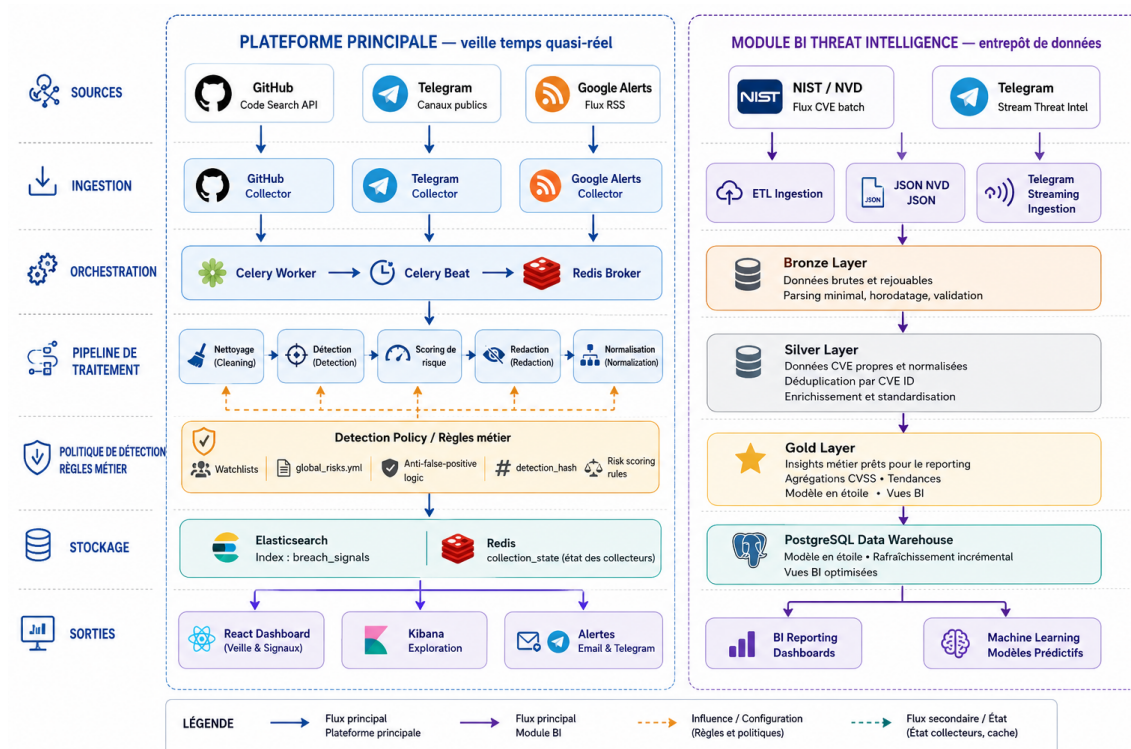


FIGURE 2.1 – Vue conceptuelle du pipeline global de veille

Plateforme principale de monitoring

La plateforme principale collecte des signaux depuis plusieurs sources OSINT, notamment GitHub, Telegram et Google Alerts. Chaque source dispose de son propre collecteur afin de séparer les responsabilités et de faciliter l'évolution du système. Après la collecte, les résultats passent par une chaîne d'analyse qui extrait les informations utiles, normalise les données, applique la *detection policy*, calcule un score de risque, attribue une sévérité et un niveau de confiance, puis redacte les données sensibles avant de stocker uniquement les signaux qualifiés.

Module BI Threat Intelligence

Le module *BI Threat Intelligence* constitue un sous-projet complémentaire. Son objectif n'est pas de détecter directement des secrets exposés, mais d'exploiter les flux **Common Vulnerabilities and Exposures** afin de produire des indicateurs décisionnels.

Ce module repose sur une architecture de type **Data Warehouse** organisée selon le modèle

médaille : Bronze, Silver et Gold. Les données CVE sont d'abord collectées à l'état brut, puis nettoyées et enrichies avant d'être transformées en tables exploitables par Power BI.

Configuration des sources

La plateforme utilise des fichiers de configuration externes pour définir les sources, les requêtes, les alertes et les règles de détection. Cette approche évite de modifier le code à chaque changement et rend la veille plus claire, maintenable et auditable.

TABLE 2.1 – Fichiers de configuration utilisés par la plateforme

Fichier	Rôle
Fichier des risques globaux GitHub	Catégories de risques et requêtes de recherche utilisées pour détecter les expositions techniques
Fichier des canaux Telegram	Liste des canaux publics suivis, catégories associées et paramètres de collecte.
Fichier des alertes Google Alerts	Alertes RSS catégorisées permettant d'assurer la veille médiatique et OSINT.
Fichier de <i>detection policy</i>	Règles de filtrage, motifs placeholders, scoring, classification et traitement des faux positifs.

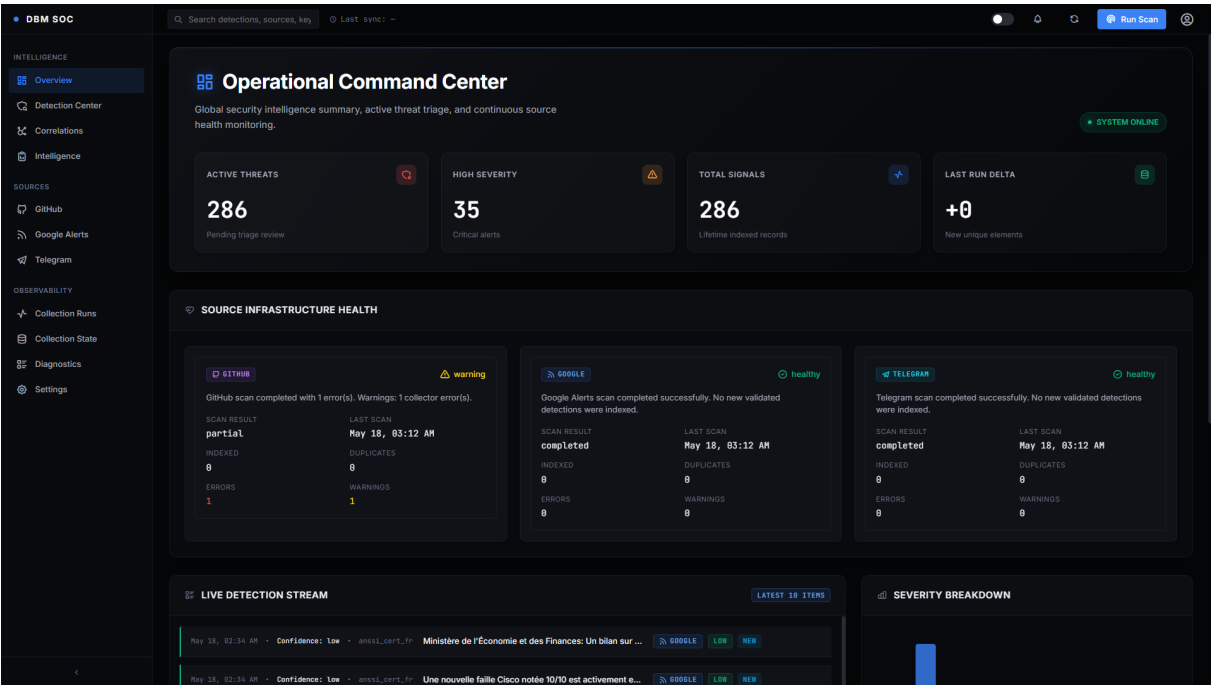
Contrôle des sources

Chaque source dispose d'un état observable dans la plateforme. Cet état permet à l'analyste de vérifier si une source fonctionne correctement ou si elle présente des erreurs.

- état technique : *healthy*, *warning*, *error*, *disabled* ;
- dernier passage de collecte et résultat associé ;
- nombre d'éléments collectés, dédupliqués et indexés ;
- compteurs d'erreurs et d'avertissements.

Ce contrôle continu permet d'expliquer pourquoi une source ne produit pas de nouvelles détections, par exemple lorsque les éléments collectés sont déjà connus ou filtrés par la politique de détection.

Data Breach & Threat Intelligence Monitoring Platform



Sources de données et stratégie de collecte

Vue d'ensemble des sources

La plateforme s'appuie sur quatre familles de sources complémentaires. Chacune apporte un type de signal différent : technique, communautaire, médiatique ou vulnérabilité.

TABLE 3.1 – Vue d'ensemble des sources intégrées dans la plateforme

Source	Type de signal	Apport principal
GitHub	Fuites techniques de secrets	Preuves d'exposition directe
Telegram	Canaux publics OSINT / CVE	Signaux communautaires précoces
Google Alerts	Veille médiatique RSS	Couverture news et OSINT
CVE Feed / BI Threat Intelligence	Vulnérabilités publiées	Vision quantitative et tendancielle

GitHub Monitoring

Rôle de la source

GitHub représente l'une des sources principales de collecte de notre plateforme, car il permet d'identifier des preuves techniques d'exposition directement présentes dans des dépôts publics. De nombreux projets publiés sur GitHub peuvent contenir, volontairement ou par erreur, des secrets applicatifs tels que des fichiers `.env`, des clés d'API, des tokens d'accès, des certificats, des identifiants de bases de données ou encore des chaînes de connexion.

Mécanisme d'accès

L'accès aux données GitHub est réalisé à travers l'API officielle **GitHub Code Search**. Un *token développeur GitHub* est utilisé afin d'authentifier les requêtes et de bénéficier de quotas plus adaptés aux opérations de recherche automatisée.

Le module exécute des requêtes ciblées sur le code public, collecte les résultats candidats, puis analyse le contenu réel des fichiers afin de vérifier si l'exposition détectée correspond réellement à un secret exploitable ou à un simple faux positif.

Organisation des requêtes

La logique de recherche GitHub repose sur un fichier de configuration appelé **global risks**. Ce fichier centralise les requêtes de détection organisées par catégories de risques.

- fichiers d'environnement : `.env`, `DATABASE_URL`, `API_KEY` ;
- identifiants de bases de données : PostgreSQL, MySQL, MongoDB, Supabase, Render ;
- clés d'API et tokens applicatifs ;
- secrets cloud : AWS, Azure, GCP ;
- secrets JWT et clés privées ;
- fichiers Docker, Kubernetes et fichiers de configuration applicative.

Chaque entrée contient une requête de recherche, une catégorie de risque et un niveau de sévérité indicatif. Cette organisation rend la veille plus structurée, auditable et évolutive.

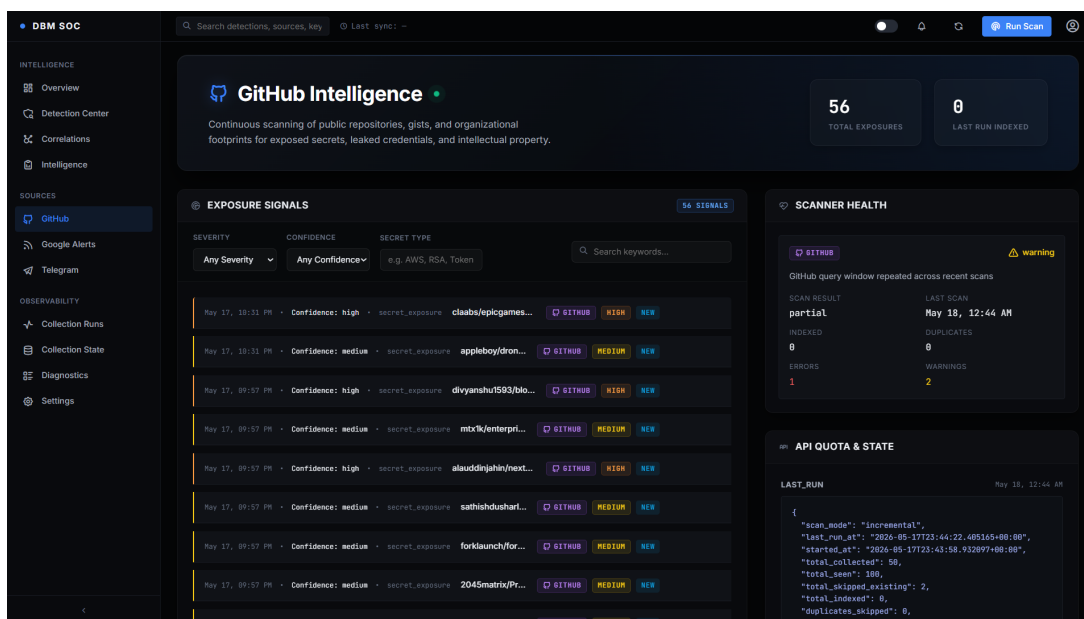


FIGURE 3.1 – Interface GitHub Intelligence affichant les expositions collectées depuis GitHub

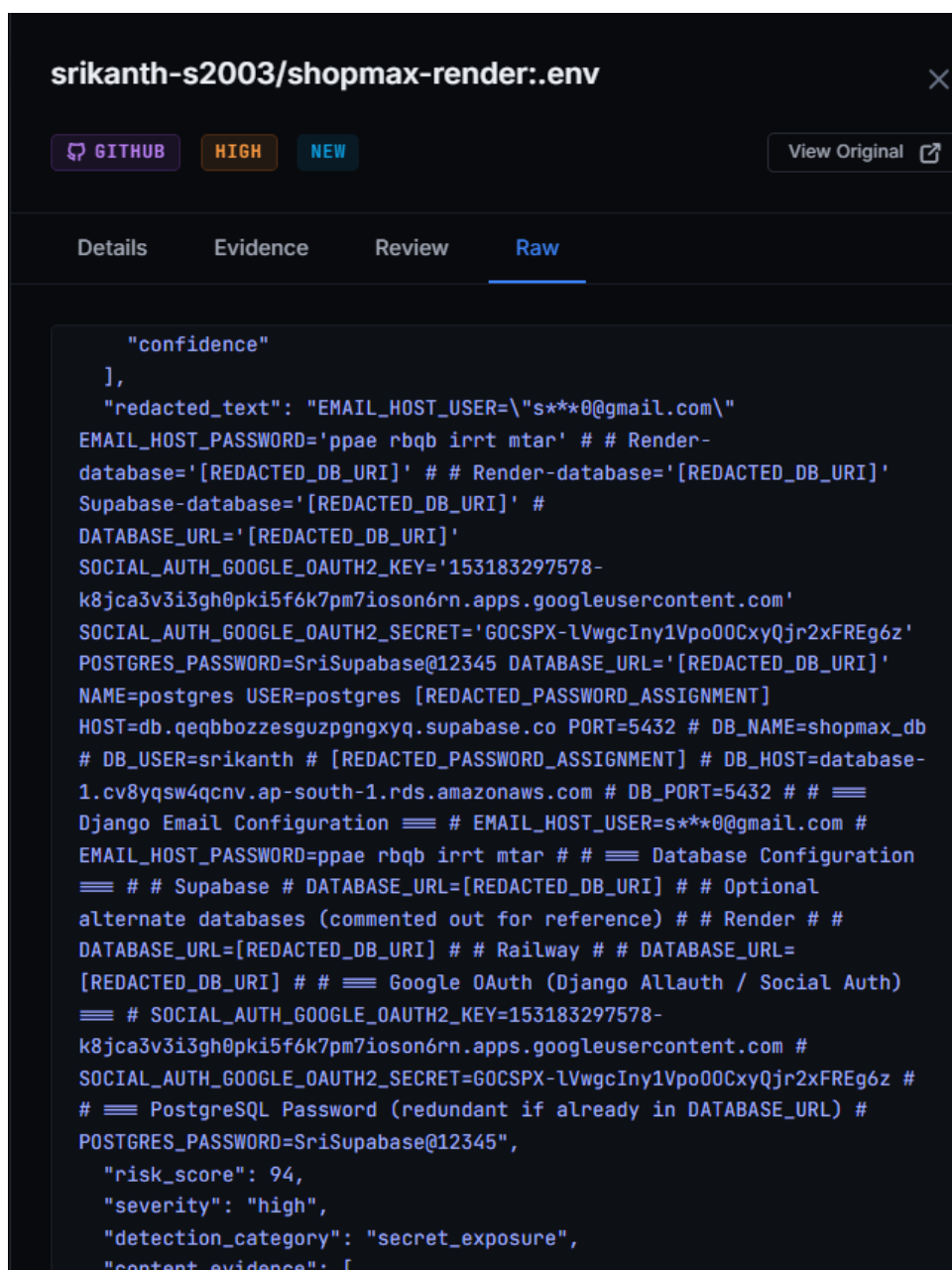
La page **GitHub Intelligence** présente les résultats collectés depuis GitHub afin de détecter les secrets, credentials et expositions publiques. Le tableau central liste les signaux détectés avec leur sévérité, leur niveau de confiance, leur type de secret et leur statut. Les panneaux latéraux affichent l'état du scanner, le dernier scan, les erreurs, les avertissements et les données indexées.

Exemple de résultat GitHub

Parmi les résultats collectés, le système a identifié un cas d'exposition critique contenant plusieurs informations sensibles dans un fichier de configuration applicatif. Le contenu détecté incluait notamment :

- des identifiants SMTP/Gmail utilisés pour l'envoi d'emails ;
- des URLs de bases de données PostgreSQL, Supabase et Render contenant des utilisateurs et mots de passe ;
- un secret Google OAuth ;
- des variables de connexion à une base de données : HOST, PORT, USER, PASSWORD.

Ce type de fuite est classé comme **secret exposure** avec une sévérité **high**, car il peut permettre à un attaquant d'accéder à des services backend, d'exploiter une base de données ou d'abuser de services tiers liés à l'application.



The screenshot shows a GitHub repository interface for the file `srikanth-s2003/shopmax-render:.env`. The repository is marked as **HIGH** severity and **NEW**. The `Raw` tab is selected, displaying the contents of the `.env` file. The file contains a JSON object with a `"confidence"` field and a `"redacted_text"` field. The `"redacted_text"` field contains a large block of text that appears to be a configuration file for a Django application, including database connection details, social authentication keys, and other settings. The text is redacted with `[REDACTED_DB_URI]` placeholders. The `"risk_score"` is 94, the `"severity"` is "high", and the `"detection_category"` is "secret_exposure".

```
"confidence": 94,
"redacted_text": "EMAIL_HOST_USER=\"s***0@gmail.com\"
EMAIL_HOST_PASSWORD='ppae rbqb irr t mtar' # # Render-
database='[REDACTED_DB_URI]' # # Render-database='[REDACTED_DB_URI]'
Supabase-database='[REDACTED_DB_URI]' #
DATABASE_URL='[REDACTED_DB_URI]'
SOCIAL_AUTH_GOOGLE_OAUTH2_KEY='153183297578-
k8jca3v3i3gh0pki5f6k7pm7ioson6rn.apps.googleusercontent.com'
SOCIAL_AUTH_GOOGLE_OAUTH2_SECRET='G0CSPX-lVwgcIny1Vpo00CxyQjr2xFREg6z'
POSTGRES_PASSWORD=SriSupabase@12345 DATABASE_URL='[REDACTED_DB_URI]'
NAME=postgres USER=postgres [REDACTED_PASSWORD_ASSIGNMENT]
HOST=db.qeqbbozzesguzpgngxyq.supabase.co PORT=5432 # DB_NAME=shopmax_db
# DB_USER=srikanth # [REDACTED_PASSWORD_ASSIGNMENT] # DB_HOST=database-
1.cv8yqsw4qcnv.ap-south-1.rds.amazonaws.com # DB_PORT=5432 # # ===
Django Email Configuration === # EMAIL_HOST_USER=s***0@gmail.com #
EMAIL_HOST_PASSWORD=ppae rbqb irr t mtar # # === Database Configuration
=== # # Supabase # DATABASE_URL=[REDACTED_DB_URI] # # Optional
alternate databases (commented out for reference) # # Render # #
DATABASE_URL=[REDACTED_DB_URI] # # Railway # # DATABASE_URL=
[REDACTED_DB_URI] # # === Google OAuth (Django Allauth / Social Auth)
=== # SOCIAL_AUTH_GOOGLE_OAUTH2_KEY=153183297578-
k8jca3v3i3gh0pki5f6k7pm7ioson6rn.apps.googleusercontent.com #
SOCIAL_AUTH_GOOGLE_OAUTH2_SECRET=G0CSPX-lVwgcIny1Vpo00CxyQjr2xFREg6z #
# === PostgreSQL Password (redundant if already in DATABASE_URL) #
POSTGRES_PASSWORD=SriSupabase@12345",
"risk_score": 94,
"severity": "high",
"detection_category": "secret_exposure",
"content_evidence": [
```

FIGURE 3.2 – Exemple d'exposition critique détectée dans un dépôt GitHub public

Telegram Monitoring

Rôle de la source

Telegram est utilisé comme une source de veille communautaire et OSINT. Certains canaux publics publient régulièrement des annonces liées aux fuites de données, aux vulnérabilités CVE, aux incidents cyber ou aux expositions revendiquées.

Contrairement à GitHub, qui fournit des preuves techniques directes à travers le code exposé, Telegram fournit principalement des indications déclaratives. Les informations collectées doivent donc être considérées comme des signaux à analyser et à valider, et non comme des preuves définitives.

Canaux suivis

La plateforme surveille actuellement deux canaux Telegram publics déclarés dans un fichier de configuration dédié.

TABLE 3.2 – Canaux Telegram publics surveillés

Canal	Catégorie	Rôle
CVEDetector	cve_intelligence	Veille sur les CVE, vulnérabilités et alertes techniques.
breachforums_cdn	data_breaches	Suivi des annonces publiques liées aux fuites de données et aux expositions revendiquées.

Logique de traitement

Les messages collectés depuis Telegram sont analysés, normalisés et transformés en signaux exploitables par le système.

- extraction des informations utiles : titre, source, date, URLs, mots-clés et identifiants CVE ;
- détection des thématiques : fuite de données, vulnérabilité, exposition, compromission ou alerte technique ;
- classification du message selon sa catégorie ;
- génération d'un signal normalisé compatible avec le reste de la plateforme ;
- masquage ou réduction du contenu afin d'éviter le stockage inutile de données sensibles.

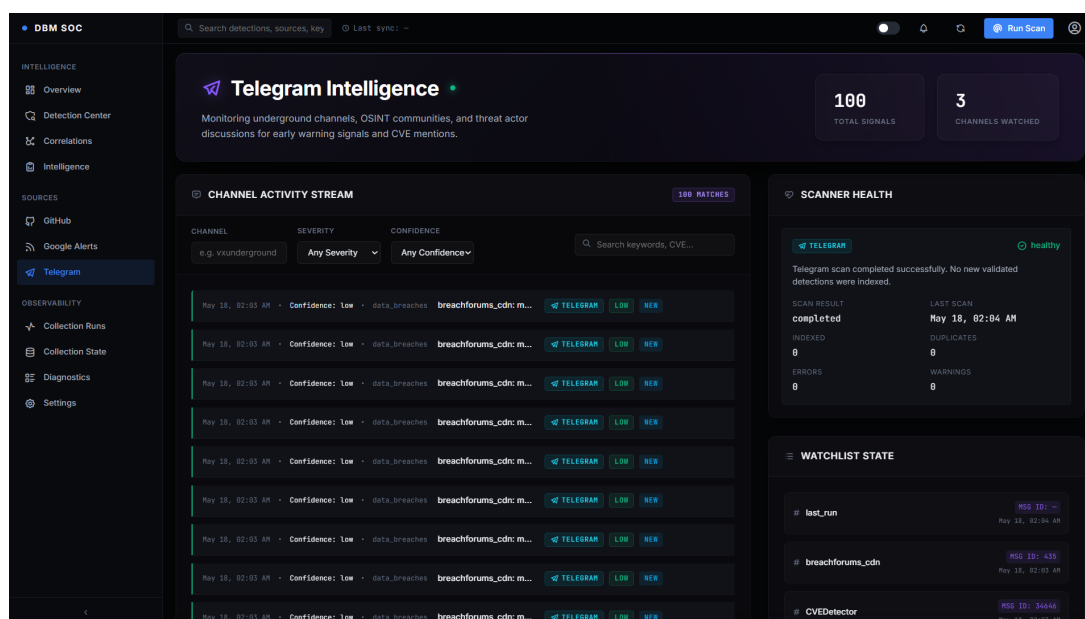


FIGURE 3.3 – Interface Telegram Monitoring affichant les signaux collectés depuis les canaux publics

Exemple de signal Telegram

Un exemple de signal collecté depuis le canal `breachforums_cdn` concerne une annonce publique mentionnant une fuite liée à **Avito.ma**. Le message indique qu'environ **2,7 millions d'enregistrements** auraient été exposés à la suite d'un incident datant approximativement de septembre 2022.

- source du signal : canal Telegram public `breachforums_cdn` ;
- organisation mentionnée : **Avito.ma** ;
- volume annoncé : environ **2,728,091 enregistrements** ;
- type de signal : annonce de fuite de données ;
- statut analytique : information à valider par corrélation avec d'autres sources.

Ce type de message ne constitue pas automatiquement une preuve technique. Il représente plutôt un signal OSINT qui doit être enrichi, vérifié et comparé avec d'autres sources avant d'être considéré comme un incident confirmé.

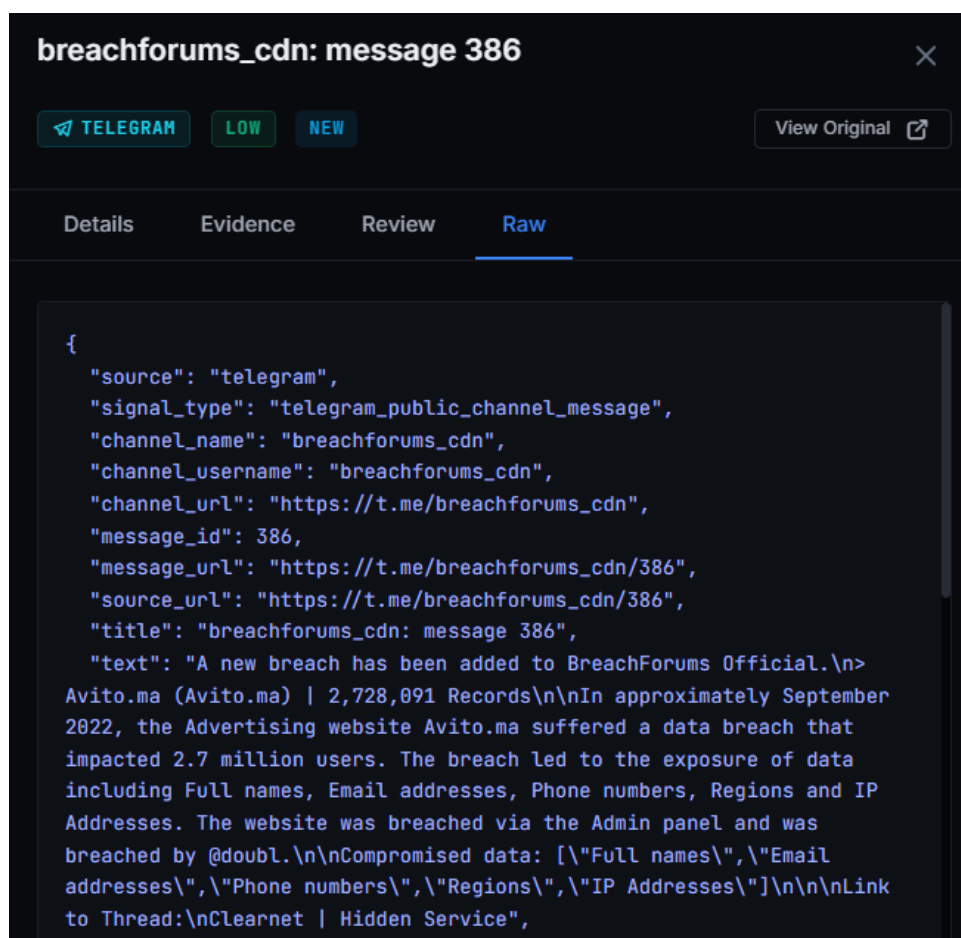


FIGURE 3.4 – Exemple de signal Telegram lié à une annonce publique de fuite de données

Google Alerts Monitoring

Rôle de la source

Google Alerts apporte la dimension **news / OSINT** au pipeline de veille. Cette source permet de capter, à grande échelle et en quasi-temps réel, la couverture médiatique des incidents de sécurité, des fuites de données et des compromissions d'entreprises.

Configuration des alertes

Au total, **16 alertes personnalisables** ont été configurées. Chaque alerte est définie par :

- un nom ;
- une catégorie ;
- un pays ou périmètre géographique ;
- une requête ciblée en français et en anglais ;
- une URL RSS produite par Google Alerts.

TABLE 3.3 – Exemples de catégories d'alertes configurées

Catégorie	Sujet surveillé
global_cyber_incidents	Cyberattaques et fuites à grande échelle
ransomware_monitoring	Incidents et groupes ransomware
healthcare_breaches	Hôpitaux et données médicales
financial_incidents	Banques, fintech, fraudes
government_incidents	Secteur public et contractants défense
zero_day_vulnerabilities	0-day et CVE activement exploités
dark_web_activity	Ventes de données, brokers d'accès
anssi_cert_fr	Bulletins officiels ANSSI / CERT-FR

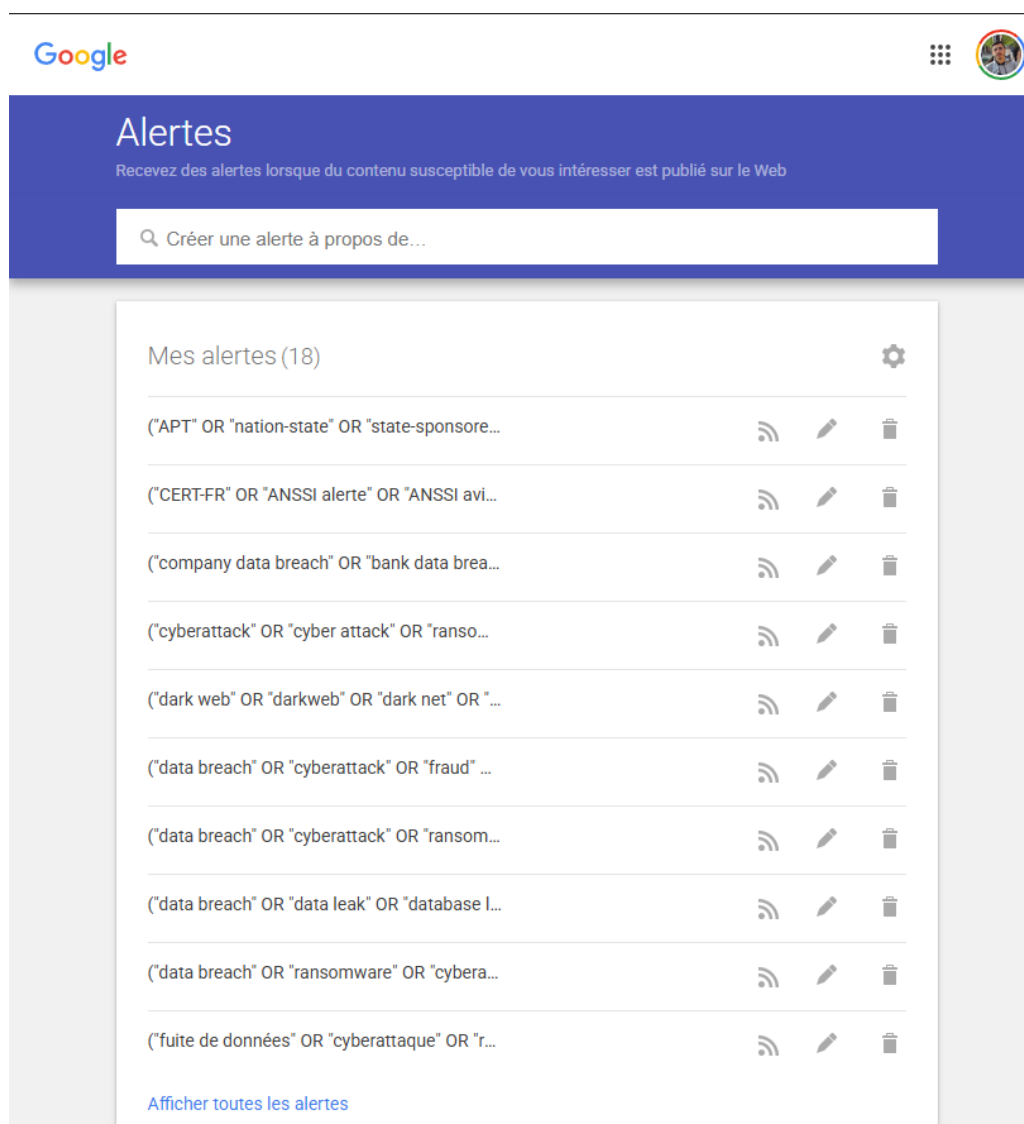


FIGURE 3.5 – image sur l'interface google alerts

Logique d'exploitation

Les flux RSS sont récupérés à intervalle régulier. Pour chaque entrée, la plateforme extrait les métadonnées, rattache la détection à sa catégorie d'origine, applique des règles de filtrage et conserve uniquement l'information normalisée et redactée.

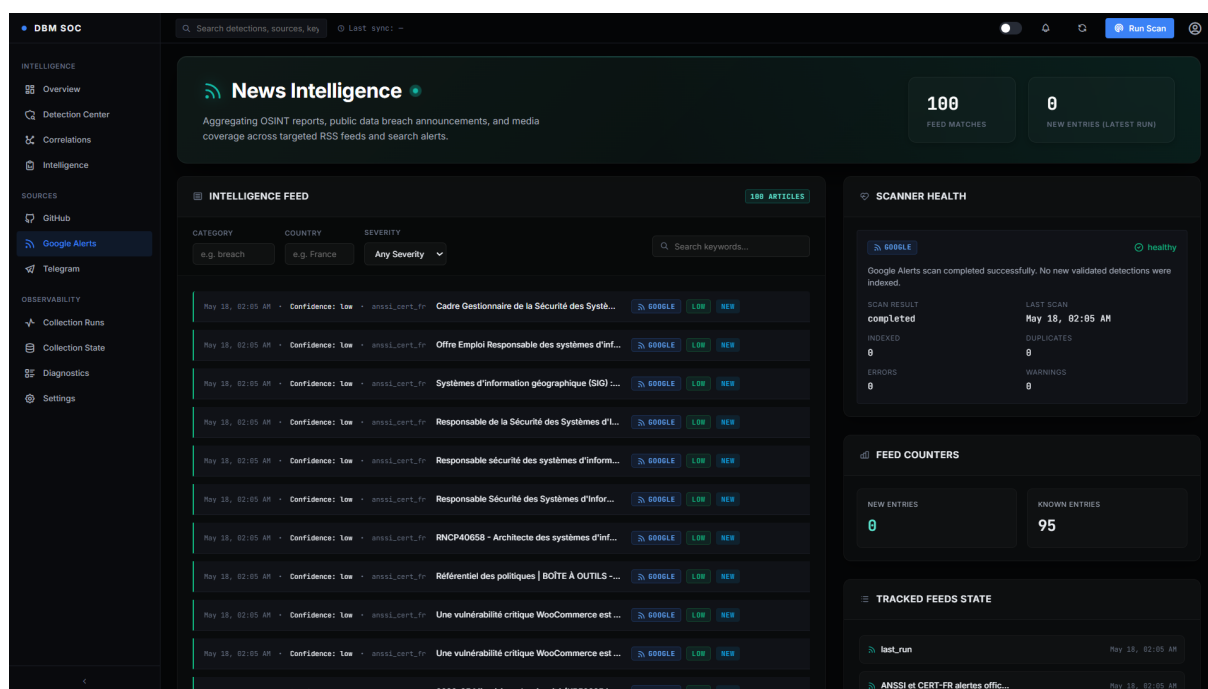


FIGURE 3.6 – Interface de suivi des alertes Google Alerts

CVE Feed et BI Threat Intelligence

Positionnement du module

Le module *BI Threat Intelligence* est un projet complémentaire à la plateforme Data Breach & Threat Intelligence Monitoring Platform. Contrairement aux modules GitHub, Telegram ou Google Alerts, qui se concentrent sur la détection d'expositions et de signaux cyber, ce module est orienté vers l'exploitation analytique des flux **Common Vulnerabilities and Exposures**.

Son objectif principal est de collecter, structurer et analyser les données CVE afin de produire une vision décisionnelle sur l'évolution des vulnérabilités dans le temps.

Architecture du pipeline CVE

Le module repose sur une architecture de type **Data Warehouse** organisée selon le modèle médaille.

— **Bronze Layer** : ingestion brute des données CVE depuis les sources externes ;

- **Silver Layer** : nettoyage, normalisation, enrichissement et déduplication ;
- **Gold Layer** : création des agrégats métiers, indicateurs de risque et tables prêtes pour Power BI.

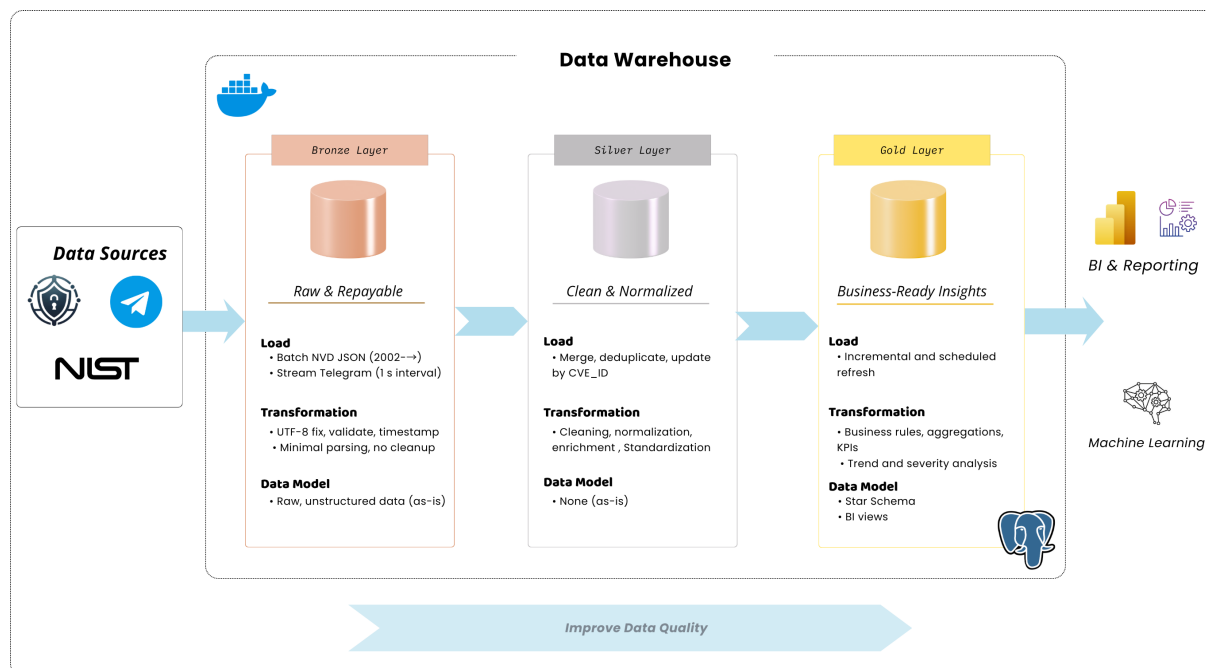


FIGURE 3.7 – Architecture du pipeline BI Threat Intelligence basée sur les couches Bronze, Silver et Gold

Rôle des tableaux de bord

La restitution constitue la dernière étape du pipeline. Elle transforme les signaux qualifiés en informations exploitables par l'analyste. Les tableaux de bord permettent de suivre les résultats collectés, l'état des sources, les erreurs, les avertissements, les tendances et les exemples de détections.

L'objectif n'est pas seulement d'afficher des données, mais de fournir une interface de revue permettant d'identifier rapidement les signaux importants.

Tableaux de bord Power BI

Le module *BI Threat Intelligence* fournit trois pages principales dans le dashboard Power BI. Chaque page répond à un objectif analytique différent.

Executive Overview

La page **Executive Overview** donne une vision globale de l'état des vulnérabilités. Elle présente les principaux indicateurs, notamment le nombre total de CVE, le volume de vulnérabilités critiques, les fournisseurs impactés et le pourcentage de CVE critiques.

Elle permet également de visualiser l'évolution annuelle des CVE, les fournisseurs les plus concernés et la répartition des vulnérabilités par niveau de sévérité.

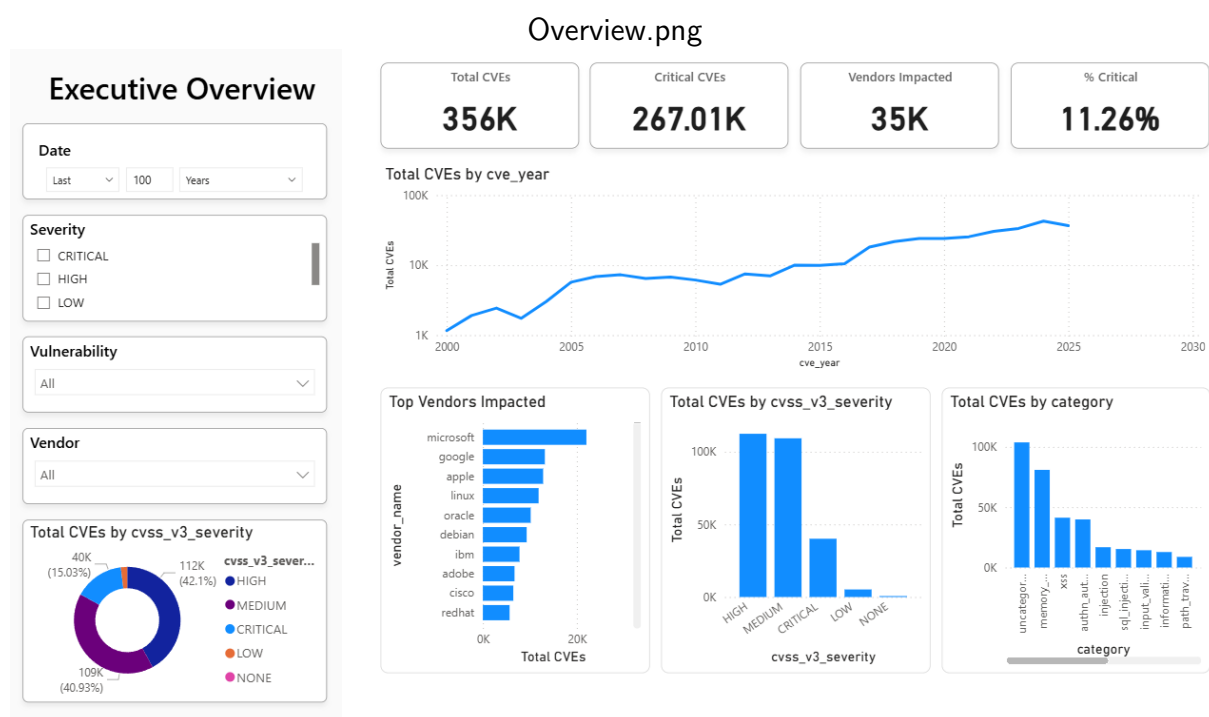


FIGURE 3.8 – Dashboard Power BI – Executive Overview des vulnérabilités CVE

Time and Age Analysis

La page **Time and Age Analysis** analyse les CVE selon leur dimension temporelle. Elle permet d'observer l'évolution du nombre de vulnérabilités par année, les tendances mensuelles, ainsi que la distribution des CVE selon leur ancienneté.

Cette page aide à comprendre si les vulnérabilités sont principalement récentes ou anciennes, et comment les niveaux de sévérité se répartissent selon l'âge des CVE.

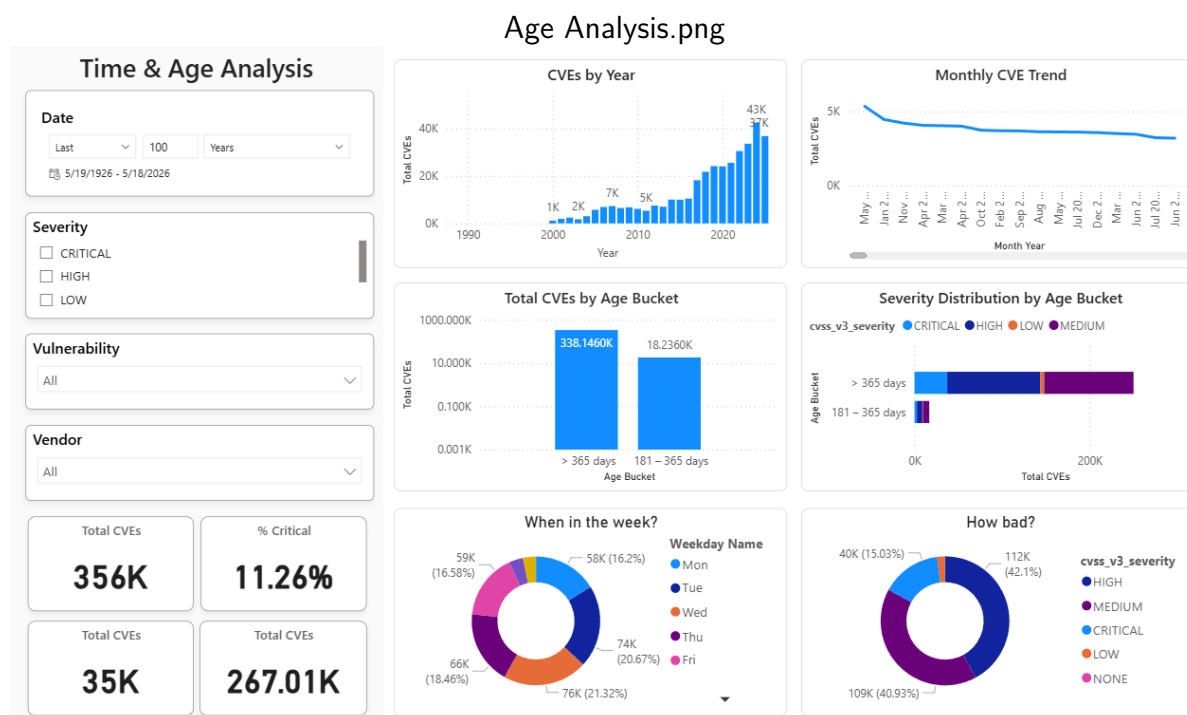


FIGURE 3.9 – Dashboard Power BI – Analyse temporelle et ancienneté des CVE

Severity and Impact Analysis

La page **Severity and Impact Analysis** se concentre sur l'analyse de la criticité et de l'impact des vulnérabilités. Elle présente la distribution des sévérités, les scores CVSS moyens et maximums, ainsi que la relation entre impact et exploitabilité.

Cette page permet d'identifier les vulnérabilités les plus critiques et d'avoir une lecture plus fine du risque à travers les métriques CVSS.

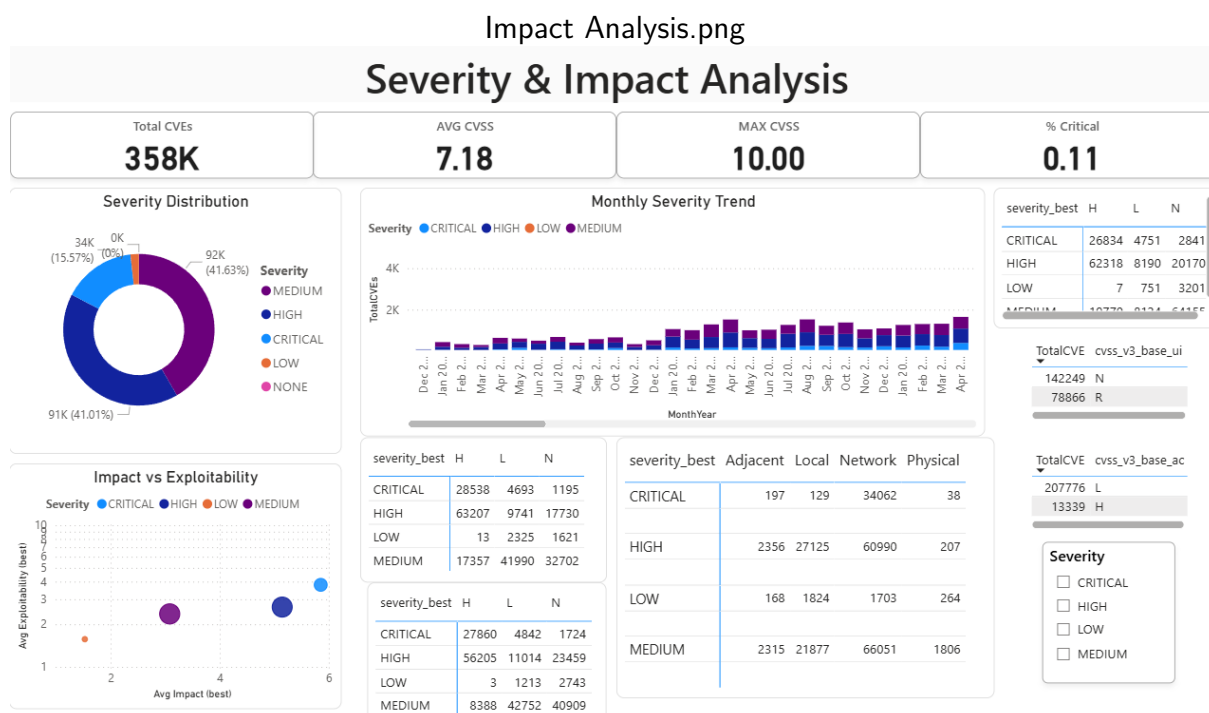


FIGURE 3.10 – Dashboard Power BI – Analyse de la sévérité et de l'impact des CVE

Apport analytique du module BI

Le module CVE Feed et BI Threat Intelligence transforme les flux CVE en indicateurs décisionnels. Il permet à l'analyste de comprendre l'évolution des vulnérabilités, d'identifier les tendances importantes et de mieux prioriser les risques.

Cette brique apporte une dimension analytique au projet global. Elle ne détecte pas seulement des signaux cyber, mais fournit une vision historique, statistique et stratégique de l'écosystème des vulnérabilités.

Synthèse des résultats obtenus

Le projet permet de centraliser plusieurs types de signaux cyber dans un même environnement. GitHub apporte les preuves techniques d'exposition, Telegram fournit des signaux communautaires précoces, Google Alerts ajoute une couverture médiatique OSINT, et le module CVE offre une lecture quantitative des vulnérabilités.

La combinaison de ces sources permet de construire une démarche de veille plus complète,

dans laquelle chaque signal peut être collecté, qualifié, visualisé et exploité par un analyste.

Traitement, détection et classification des risques

Méthodologie de collecte

La collecte s'opère selon deux régimes complémentaires.

- **Collecte planifiée** : un planificateur exécute périodiquement les collecteurs afin d'assurer une couverture continue.
- **Collecte à la demande** : l'analyste peut déclencher un scan immédiat depuis l'interface pour valider un changement de configuration ou enquêter sur un signal.

Limites de volumétrie

Pour préserver la stabilité du service et respecter les quotas externes, chaque source applique des plafonds de volumétrie configurables : nombre maximal de requêtes par exécution, nombre maximal d'éléments par requête, nombre maximal de récupérations de contenu et taille maximale du contenu téléchargé.

Cette discipline est essentielle pour transformer une collecte ponctuelle en une chaîne de veille soutenable dans la durée.

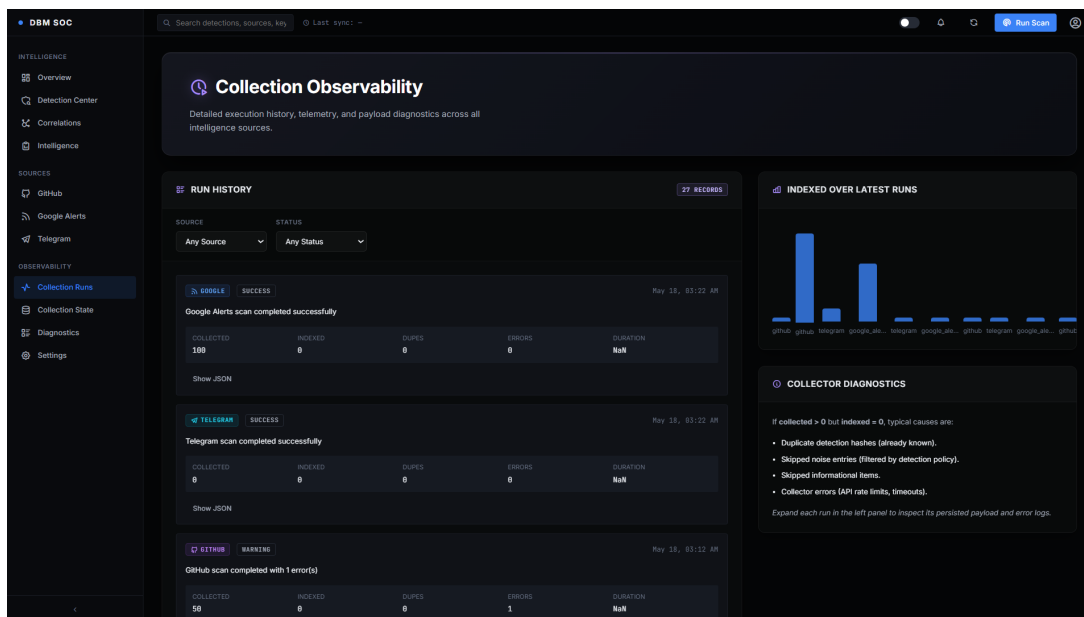
Déduplication

Toute détection est associée à un **hash de déduplication**. Lorsqu'un nouveau passage de collecte rapporte une donnée déjà connue, la plateforme l'ignore avant l'indexation.

Ce mécanisme évite la pollution de la base de détections et explique pourquoi le total indexé peut rester stable malgré une activité de collecte réelle.

Vue d'ensemble des passages de collecte

Chaque exécution est enregistrée dans une trace dédiée appelée *collection run*. Cette trace détaille la source concernée, la date du passage, sa durée, le nombre d'éléments collectés, indexés, dédoublés, ainsi que les erreurs et avertissements éventuels.



Normalisation des données

Les sources ne produisent pas le même type de données : GitHub retourne du code, Telegram du texte de message, Google Alerts des entrées RSS et les flux CVE des objets structurés. Une étape de normalisation est donc nécessaire.

Cette étape transforme chaque résultat brut en un document de détection unifié comportant des champs communs : source, URL publique, titre, organisation éventuellement concernée, catégorie de risque, indicateurs détectés, texte redacté, score, sévérité, statut analyste et horodatages.

Analyse du contenu

L'analyse de contenu vise à extraire les indicateurs qui donnent de la valeur à une détection. Selon la source, il peut s'agir :

- de motifs de secrets dans du code ;
- de mentions explicites de fuites de données ;
- de mentions d'organisations victimes ;
- de références CVE ;
- de métadonnées liées à la source ou à la catégorie d'alerte.

Filtrage

Le filtrage applique plusieurs catégories de règles complémentaires.

- **Filtres de valeurs placeholders** : élimination des chaînes comme *changeme*, *your_key*, *example* ou *xxx*.
- **Filtres d'emplacement** : exclusion des dossiers et fichiers manifestement d'exemple comme *tests/*, *docs/*, *examples/* ou *.env.example*.
- **Filtres de score** : exigence d'un score minimum pour qu'une détection puisse être indexée.

Detection policy

La *detection policy* est le garde-fou central de la plateforme. Elle décide si une donnée brute mérite d'être conservée et détermine sa sévérité.

Elle permet notamment de :

- vérifier le contenu réel du fichier et non uniquement son nom ;
- identifier les secrets potentiellement exploitables ;
- exclure les faux positifs fréquents ;
- réduire la confiance des chemins d'exemple ;
- calculer un score de risque ;
- associer une sévérité au signal ;
- masquer les valeurs sensibles avant stockage.

Classification des risques

Chaque détection retenue est classée selon plusieurs axes.

1. **Catégorie de risque** : `env_files`, `database_credentials`, `api_keys`, `cloud_keys`, `jwt_tokens`, `private_keys`, `docker_secrets`, etc.
2. **Niveau de criticité** : `low`, `medium`, `high`.
3. **Niveau de confiance** : `low`, `medium`, `high`.
4. **Statut analyste** : `new`, `reviewed`, `false_positive`, `confirmed`, `escalated`.

TABLE 4.1 – Axes de classification des détections

Axe	Valeurs possibles
Source	<code>github</code> , <code>telegram</code> , <code>google_alerts</code> , <code>mock_paste</code>
Catégorie de risque	<code>env_files</code> , <code>database_credentials</code> , <code>api_keys</code> , <code>cloud_keys</code> , <code>jwt_tokens</code> , <code>private_keys</code> , <code>docker_secrets</code> , <code>framework_configs</code>
Sévérité	<code>low</code> , <code>medium</code> , <code>high</code>
Confiance	<code>low</code> , <code>medium</code> , <code>high</code>
Statut analyste	<code>new</code> , <code>reviewed</code> , <code>ignored</code> , <code>confirmed</code> , <code>false_positive</code> , <code>escalated</code>

Gestion des faux positifs

La plateforme ne considère pas automatiquement chaque résultat brut comme un incident. Certains contenus peuvent ressembler à des secrets sans être réellement exploitables. C'est le cas des fichiers d'exemple, des valeurs placeholders ou des contenus volontairement fictifs.

La détection initiale reste utile, car elle identifie un candidat. Cependant, l'analyse de contenu et la *detection policy* permettent de distinguer un vrai secret exploitable d'un contenu de test ou d'un faux positif.

Conclusion générale

Le projet *Data Breach & Threat Intelligence Monitoring Platform* a permis de concevoir et de mettre en œuvre une plateforme de veille cyber cohérente, qui traite la veille comme un processus méthodologique et non comme une simple accumulation de logs.

À travers quatre sources complémentaires — GitHub pour les preuves techniques, Telegram pour le signal communautaire, Google Alerts pour la dimension médiatique, et le module BI Threat Intelligence pour les vulnérabilités CVE — la plateforme couvre un large spectre d'expositions publiques et de signaux cyber.

La méthodologie repose sur une séparation claire entre la collecte, l'analyse, la classification et la restitution. Cette séparation donne au système une meilleure lisibilité, une meilleure maintenabilité et une capacité d'évolution plus importante.

Au-delà de l'apport technique, ce projet constitue un exercice méthodologique. Il montre comment, en s'appuyant sur des sources ouvertes et autorisées, il est possible de construire une chaîne de veille rigoureuse, défensive et exploitable, en cohérence avec les exigences académiques d'un projet de fin d'études.

Les perspectives identifiées, comme l'amélioration de la corrélation, l'automatisation des rapports et l'enrichissement des indicateurs, ouvrent la voie à un passage progressif d'un outil pédagogique vers un véritable outil opérationnel.